

Deep Learning Hyperparameter Optimization for IPL Match Winner Prediction

Final Project Report

Jagdeep

PG Diploma in AI & Applied Machine Learning | Fanshawe College
April 2026 | Deep Learning Optimization Course

Abstract

This report compares Random Search and Bayesian Optimization (Optuna/TPE) for tuning a Keras MLP that predicts IPL match winners. The dataset has 328 matches with pre-match features only, and both methods ran 50 trials over an identical search space. Random Search hit the best test accuracy (54.3%), Optuna produced the best F1 (0.578) and recall (63.5%), and — somewhat unexpectedly — the untuned baseline led on ROC-AUC (0.566) and log loss (0.689). The key takeaway: optimization shifted the precision-recall tradeoff rather than producing a universally better model. SHAP analysis revealed season year and match month as dominant features, with toss winner's prior win rate as the strongest cricket-specific signal.

1. Introduction

The IPL is one of the biggest T20 cricket leagues globally, and predicting match outcomes is genuinely hard. Results depend on team form, venue, toss decisions, and seasonal dynamics — all of which need to be captured using only pre-match information, since no in-game data exists at prediction time. The class balance is nearly even (51.8% majority class), so a naive predictor barely helps. It's a challenging tabular classification task, and that's what makes it useful for studying optimization.

Neural networks are sensitive to hyperparameters, and the choice of optimizer, learning rate, dropout, and network width doesn't just affect accuracy — it shifts the entire precision-recall profile. This project has three goals: build a clean, leakage-free IPL prediction pipeline; compare Random Search and Bayesian Optimization head-to-head with the same 50-trial budget; and analyze how optimization actually changes model behavior — backed by SHAP explainability.

2. Methodology

2.1 Data Preprocessing

The dataset comes from Kaggle — IPL match-level data from 2008–2024, with 1,090 raw records. After filtering incomplete matches and engineering historical features, 328 remained. The target is binary: did Team 1 win? Leakage control was strict — every post-match column (winner, Man of the Match, result margin) was dropped. Only pre-match features were kept: season year, match month, toss winner/decision, city, venue, and engineered features like each team's prior win rate and head-to-head record (computed chronologically so no future data leaks in).

Preprocessing: franchise name standardization (e.g., Delhi Daredevils → Delhi Capitals), one-hot encoding for categoricals, StandardScaler on numeric features, and a

chronological 70/15/15 train/val/test split — no random shuffling, simulating real forecasting.

2.2 Model Architecture

The base model is a Keras MLP: Input $\rightarrow$ Dense(64, ReLU) $\rightarrow$ Dropout(0.2) $\rightarrow$ Dense(32, ReLU) $\rightarrow$ Dropout(0.2) $\rightarrow$ Dense(1, Sigmoid). Loss: binary cross-entropy. Optimizer: Adam (lr=1e-3). Training: up to 60 epochs with early stopping on validation AUC (patience=5). Evaluated on accuracy, F1, ROC-AUC, precision, recall, and log loss.

2.3 Hyperparameter Optimization

Both methods share the exact same search space and 50-trial budget:

Hyperparameter	Search Range	Type
Learning Rate	1e-4 — 3e-2	Log-uniform
Batch Size	{16, 32, 64, 128}	Categorical
Dropout Rate	{0.0, 0.1, 0.2, 0.3, 0.4, 0.5}	Categorical
Hidden Units	{32, 64, 128, 256}	Categorical
Optimizer	{Adam, RMSProp, SGD}	Categorical

Table 1: Shared hyperparameter search space. Seed = 42 for reproducibility.

Baseline Method: Random Search. Samples hyperparameters uniformly at random — simple, parallel, broad coverage. Learning rate sampled log-uniformly; everything else categorical. 50 trials, completed in 173.2 seconds.

Bayesian Optimization (Optuna/TPE). Uses Optuna's Tree-structured Parzen Estimator to model which regions produce good vs. poor results, then samples more from promising areas. Each trial informs the next — guided, sequential search. 50 trials, 188.0 seconds.

Comparison of Results (Performance + Runtime):

	Random Search	Bayesian (Optuna)
Trials / Runtime	50 / 173.2 s	50 / 188.0 s
Best Val Accuracy	0.6012	0.5644
Best Val ROC-AUC	0.5792	0.5694
Learning Rate	4.56×10^{-4}	1.97×10^{-4}
Dropout / Units	0.50 / 64	0.20 / 256
Optimizer	RMSProp	SGD

Table 2: Validation results and best hyperparameters found by each method.

Random Search edged out on validation accuracy and ROC-AUC with comparable runtime. The two methods found fundamentally different architectures — Random Search went lean and regularized (heavy dropout, small network, RMSProp), while Optuna went wide and aggressive (light dropout, 256 units, SGD). Both agreed on batch size 128. The convergence plots below show Optuna’s TPE found its best region early (~trial 10), while Random Search improved gradually with a jump near trial 35.

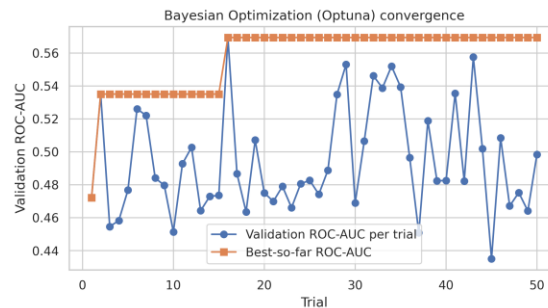


Figure 1: Bayesian Optimization (Optuna) — validation ROC-AUC convergence over 50 trials.

2.4 Optimizer & Regularization Choices

An ablation study isolated optimizer effects by fixing everything at Optuna’s best config and varying only the optimizer. Adam got the best accuracy (0.564) with low recall (0.222). SGD had the highest recall (0.653) but lowest accuracy (0.485). RMSProp sat in between with the best ROC-AUC (0.551) but very low recall (0.208). Optimizer choice isn’t just convergence speed — it directly shifts the precision-recall tradeoff.

Regularization had a similarly dramatic effect. With zero regularization, recall spiked to 91.7% but accuracy dropped to 47.2% — the model predicted positive for almost every input. Dropout alone restored balance (accuracy 53.4%, recall 30.6%). The full stack (dropout + weight decay + LR schedule) pushed to 49.7% accuracy with 73.6% recall. Regularization acts as a precision-recall lever — Random Search’s heavy dropout (0.50) and Optuna’s light dropout (0.20) represent fundamentally different prediction philosophies.

2.5 Training Process & Challenges

The dataset is small — ~230 training samples after splitting. Overfitting happens fast, making early stopping on validation AUC (patience=5) essential. One-hot encoding blew up the feature space with sparse columns, and optimizer sensitivity was extreme — switching Adam to SGD (same everything else) flipped the model from precision-heavy to recall-heavy. Tracking both accuracy and recall throughout tuning was necessary; a single metric would have been misleading. Seed 42 was fixed across NumPy, TensorFlow, and Optuna. Final models were retrained on combined train+validation data before test evaluation.

3. Results & Evaluation

All four models were evaluated on the held-out test set (newest 15%):

Model	Acc	F1	Prec	Recall	AUC	LogLoss
Simple MLP Baseline	0.524	0.361	0.595	0.259	0.566	0.689
Best Optuna MLP	0.518	0.578	0.529	0.635	0.528	0.692
Best Random Search MLP	0.543	0.540	0.564	0.518	0.513	0.695
Dummy Baseline	0.518	0.683	0.518	1.000	0.500	7.764

Table 3: Test set results across all four models, sorted by ROC-AUC.

No single model dominated all metrics. The untuned baseline had the best ROC-AUC (0.566) and log loss (0.689) — best at ranking and calibrating probabilities. Random Search got the highest accuracy (54.3%). Optuna had the best F1 (0.578) and recall (63.5%) — it caught more Team 1 wins but at the cost of more false positives.

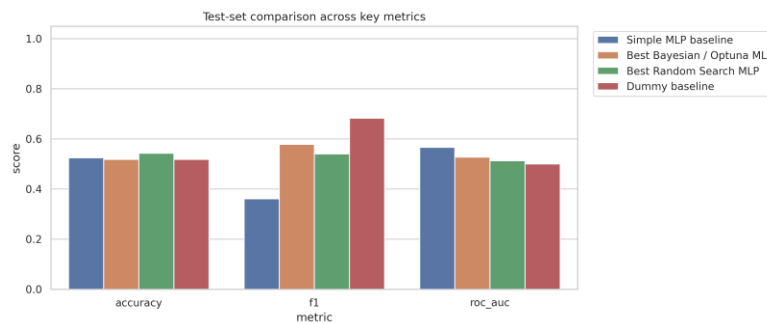


Figure 2: Test set comparison across key metrics for all four models.

The confusion matrices show the behavioral difference. Random Search predicted 78 positives (TP=44, FP=34, TN=45, FN=41). Optuna was much more aggressive: 102 positives (TP=54, FP=48, TN=31, FN=31) — the lighter dropout and bigger network pushing the decision boundary. On the ROC curves, the baseline showed the widest separation from the diagonal (AUC=0.57), followed by Optuna (0.53) and Random Search (0.51).

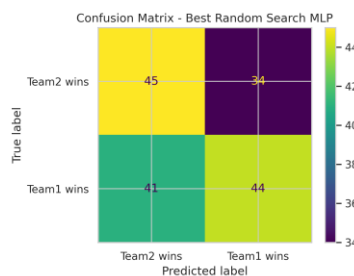


Figure 3a: Confusion Matrix — Random Search MLP.

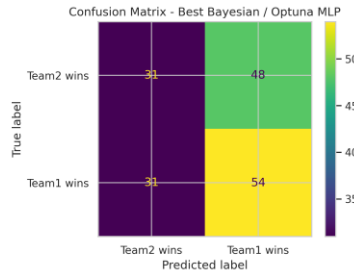


Figure 3b: Confusion Matrix — Optuna MLP.

Venue-level error analysis revealed Delhi's Arun Jaitley Stadium as the hardest ground (66.7% error over 12 matches). The model did best at Dharamsala (75%, 4 matches) and Jaipur (70%, 10 matches). Some venues likely have characteristics — pitch type, altitude, crowd dynamics — that current features don't capture.

4. Explainability

4.1 Global Explanation

SHAP was applied to the best test-set model. The global importance ranking:

Rank	Feature	Mean SHAP
1	season_start_year	0.0248
2	match_month	0.0090
3	toss_winner_prior_win_rate	0.0031
4	team1_is_toss_winner	0.0030
5	team2_prior_win_rate	0.0019
6	toss_decision_bat	0.0016

Table 4: Top 6 features by mean absolute SHAP value.

season_start_year dominates at 0.0248 — nearly 3x the next feature. The model leans on temporal trends (rule changes, franchise maturity) more than cricket-specific signals. That's a generalization concern — it's a temporal proxy, not a domain predictor. match_month ranks second (0.0090), likely reflecting scheduling or weather effects. The top cricket-domain feature is toss_winner_prior_win_rate (0.0031) — the toss winner's historical form is the strongest sport-specific signal.

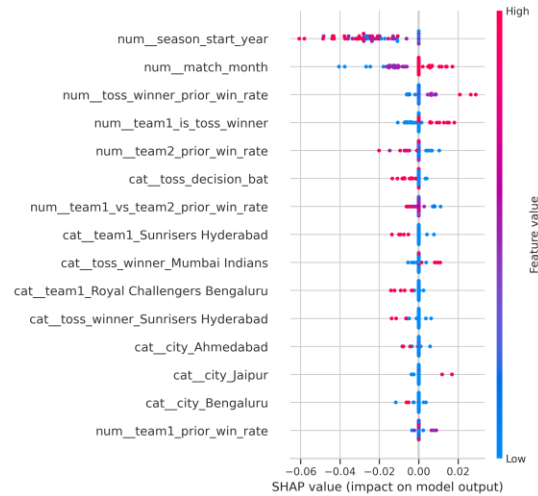


Figure 4: SHAP global summary (beeswarm). Red = high feature value, blue = low.

4.2 Local Explanation

A SHAP waterfall for one test match showed $f(x) = 0.504$ — essentially a coin flip. Season year pushed the prediction down (-0.01), toss winner's prior win rate pushed it up ($+0.01$), and team-specific features added small opposing forces that nearly cancelled out. The model correctly expressed high uncertainty when features disagreed — sensible behavior for a noisy task.

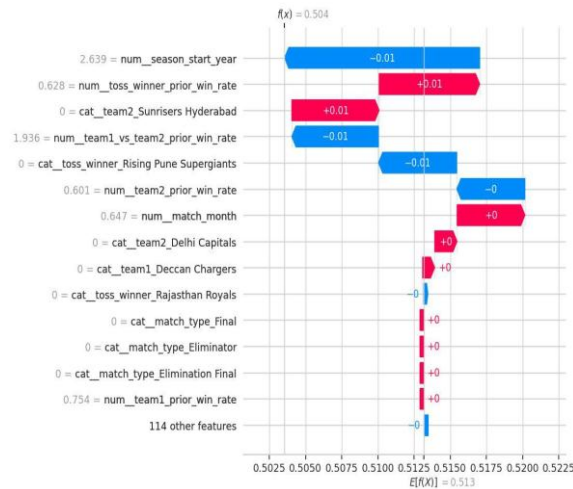


Figure 5: Local SHAP waterfall — individual match prediction ($f(x) = 0.504$).

5. Conclusion & Future Work

This project compared Random Search and Bayesian Optimization for tuning a Keras MLP on IPL match prediction. The two methods found fundamentally different architectures — one lean and regularized, one wide and aggressive. No single model dominated all test metrics. The central finding: optimization changes model behavior,

not just accuracy. It shifts the precision-recall tradeoff, and understanding that tradeoff matters more than chasing a single number.

On small, low-signal tabular datasets, Bayesian Optimization doesn't automatically beat Random Search — the space is compact enough for random sampling to cover it.

Optimizer and regularization choices are precision-recall decisions, not just convergence details. SHAP is essential for catching proxy-feature reliance and guiding future feature engineering.

Next steps: richer pre-match features (player stats, form, weather, pitch reports), comparison with XGBoost/TabNet/FT-Transformer, probability calibration, venue-specific cricket features, and multi-objective optimization targeting both accuracy and calibration jointly.

6. References

IPL Match Dataset. Kaggle. <https://www.kaggle.com/datasets/> (IPL Matches 2008–2024).